

Striver SDE Sheet - Linked List Part I

Quick DSA Revision Notes

Revision format: Approach -> core idea -> time and space complexity. Focus on the optimal pattern before interviews.

1. Reverse a Linked List

Approach	Core idea	Time	Space
Brute force	Store node values in a stack and overwrite them while popping. Links remain unchanged.	$O(N)$	$O(N)$
Optimal - Iterative	Use prev, curr and next. Save curr.next, reverse curr.next to prev, then move both pointers forward. prev becomes the new head.	$O(N)$	$O(1)$
Recursive	Reverse the remaining list, set head.next.next = head, then head.next = null.	$O(N)$	$O(N)$ stack

Revision key: Key: Never change curr.next before saving the next node.

2. Find the Middle of a Linked List

Approach	Core idea	Time	Space
Brute force	Count nodes, then traverse $N/2$ steps from the head.	$O(N)$	$O(1)$
Optimal - Slow/Fast	Move slow by one and fast by two. When fast reaches the end, slow is at the middle. For even length, it returns the second middle.	$O(N)$	$O(1)$

Revision key: Pattern: Slow and fast pointers.

3. Merge Two Sorted Linked Lists

Approach	Core idea	Time	Space
Brute force	Put all values into an array, sort it and build a new list.	$O((N+M) \log(N+M))$	$O(N+M)$
Better	Compare both lists and create a new sorted list.	$O(N+M)$	$O(N+M)$
Optimal	Use a dummy node and attach the smaller existing node each time. Append the remaining list at the end.	$O(N+M)$	$O(1)$

Revision key: Key: Dummy node removes special handling for the first node.

4. Remove Nth Node from End

Approach	Core idea	Time	Space
Brute force	Count total nodes, then delete the $(N-n+1)$ th node from the beginning.	$O(2N)$	$O(1)$
Optimal - Two pointers	Use a dummy node. Move fast n steps ahead, then move fast and slow together until fast.next is null. Delete slow.next.	$O(N)$	$O(1)$

Revision key: Key: Dummy node safely handles deletion of the head.

5. Add Two Numbers

Approach	Core idea	Time	Space
Optimal	Traverse both lists together. $sum = carry + available\ digits$; $digit = sum \% 10$; $carry = sum / 10$. Continue while a node or carry remains.	$O(\max(N, M))$	$O(\max(N, M))$ answer

Revision key: Key: Use a dummy node and do not forget the final carry.

6. Delete a Given Node

Approach	Core idea	Time	Space
Optimal	Copy the next node value into the current node, then bypass the next node: $node.next = node.next.next$.	$O(1)$	$O(1)$

Revision key: Limitation: The last node cannot be deleted using this method.